1. Visão Geral e Contexto do Projeto

No ambiente de Bug Bounty e testes de segurança autorizados, o reconhecimento (Recon) é fundamental. Processos manuais tendem a ser demorados, propensos a falhas humanas e difíceis de escalar em ativos de grande porte (*Large Scope*). A automação surge para padronizar a coleta de dados e permitir que o pesquisador foque no raciocínio crítico e na análise profunda de vulnerabilidades.
Este documento tem como objetivo estruturar estratégias de automação, analisar soluções Open Source consagradas no mercado, mapear a utilização de APIs de IA (Tier Gratuito) e avaliar a viabilidade de execução de modelos locais (*Local AI*) em arquiteturas Apple Silicon (MacBook Pro com 16 GB de RAM Unified).

2. Análise de Arquitetura e Ferramentas Open Source

2.1 Efetividade e Estágios do Recon

Um pipeline eficiente de Recon modulariza cada etapa do processo:

1. **Descoberta de Ativos (Asset Discovery):**
    - *Amas, Subfinder, Assetfinder, Chaos, Findomain*
2. **Resolução DNS e Probing de Portas:**
    - *Massdns, puredns, Naabu, Masscan*
3. **Detecção de Serviços HTTP/Web:**
    - *httpx, httprobe*
4. **Mapeamento de Conteúdo e Crawling:**
    - *katana, gau (GetAllUrls), waybackurls, Hakrawler*
5. **Análise de Parâmetros e Configurações:**
    - *Arjun, ffuf, Nuclei (para fingerprinting e checagem de misconfigurations)*

2.2 Estruturas de Automação (Frameworks Existentes)

Antes de construir uma solução do zero, vale analisar a abordagem de ferramentas consolidadas:

- **Axiom:** Utiliza infraestrutura em nuvem sob demanda (frotas de VPS) para paralelizar tarefas do Recon (distribuindo varreduras do massdns ou ffuf).
- **Recon-ng:** Estrutura modular em Python no estilo Metasploit, focada em OSINT e coleta organizada em banco de dados.
- **Osmedeus / reNgine:** Frameworks com interface web (UI) e orquestração de workflows para automação contínua.

3. Integração com Inteligência Artificial (Visão de Futuro)

A integração de IA no Recon não deve visar a substituição do pensamento crítico, mas sim a **filtragem de ruído** e **priorização de contexto**.

3.1 Casos de Uso Viáveis para IA em Recon

- **Triagem e Priorização de URLs:** Analisar listas extensas oriundas do *katana* ou *gau* e categorizar endpoints críticos (ex: painéis administrativos, APIs sem documentação, endpoints com Upload).
- **Análise de Código JavaScript:** Identificação de rotas ocultas, parâmetros não documentados e potenciais segredos em arquivos .js estáticos.
- **Resumo de Respostas HTTP:** Identificação de comportamentos anômalos em cabeçalhos ou respostas de erro.

3.2 Utilização de APIs (Free Tiers)

Principais considerações ao integrar provedores na nuvem:

- **Google Gemini (Gemini Flash):** Excelente para janelas de contexto massivas (ideal para passar grandes volumes de outputs de Recon ou arquivos JS extensos) com cotas gratuitas generosas.
- **Groq / Cerebras API:** Provedores de inferência ultra-rápida utilizando modelos open-weight (como Llama 3/3.1). Ideais para tarefas que exigem baixa latência.
- **OpenAI (GPT-4o mini) / Anthropic (Claude Haiku):** Excelentes taxas de raciocínio para análise sintática e classificação estruturada em JSON.

4. Viabilidade de IA Local (Local LLM no MacBook Pro - 16 GB RAM)

Com a arquitetura de memória unificada dos chips Apple Silicon, é perfeitamente viável executar modelos de linguagem diretamente no sistema sem custos de API.

4.1 Limitações e Capacidade de Hardware (16 GB Unified Memory)

- A memória é compartilhada entre o Sistema Operacional, aplicações e a VRAM da GPU.
- **Tamanho Máximo Recomendado para Modelos:** Modelos de **7B a 14B parâmetros** com quantização de 4-bit (Q4_K_M) ou 8-bit (Q8_0).
- **Consumo Estimado:**
    - Modelos 7B/8B (ex: Llama 3.1 8B, Qwen 2.5 7B): Consomem entre 4.5 GB e 6 GB de RAM.
    - Modelos 14B (ex: Qwen 2.5 14B Q4): Consomem entre 8 GB e 10 GB de RAM.

4.2 Engines e Ferramentas para Execução Local

- **Ollama:** A solução mais simples para execução em background via CLI ou API REST local (localhost:11434).
- **LM Studio:** Interface gráfica excelente para testes rápidos de modelos e benchmarking.
- **llama.cpp:** Implementação nativa em C/C++ com aceleração por Metal (GPU Apple), oferecendo o máximo desempenho e menor consumo de recursos.

4.3 Modelos Recomendados para Tarefas de Recon

1. **Qwen 2.5 (7B / 14B - Coder ou Instruct):** Altíssimo desempenho em análise de código, estruturação de dados em JSON e interpretação de texto.
2. **Llama 3.1 (8B):** Excelente para raciocínio geral e tarefas de classificação.
3. **DeepSeek-Coder (7B):** Muito eficiente na leitura e parsing de arquivos de código (JavaScript, Python, etc.).

5. Roteiro de Desenvolvimento (Roadmap Sugerido)

Fase 1: Padronização e Estruturação do Script (Atual)

- Garantir que as saídas dos comandos passem por *sanitização* e sejam salvas em formatos estruturados (JSON, CSV).
- Tratar duplicatas e aplicar deduplicação eficiente (anew, sort -u).

Fase 2: Modularização e Orquestração

- Permitir a execução de módulos independentes (ex: --only-dns, --only-http, --full).
- Adicionar gerenciamento de logs e tratamento de erros para interrupções inesperadas.

Fase 3: Integração Híbrida com IA

- Criar um módulo opcional de pós-processamento via API/Local AI.
- Exemplo de fluxo:
    1. Script coleta arquivos .js.
    2. Módulo de IA envia o conteúdo para o modelo local (Qwen 2.5 7B via Ollama).
    3. O modelo retorna uma lista filtrada de rotas de API encontradas.

6. Conclusão e Boas Práticas

Automações de Recon devem focar na eficiência operacional sem sobrecarregar a infraestrutura de destino ou a máquina local. A combinação de ferramentas consolidadas em ecossistema Go/C com a análise inteligente via LLMs locais oferece um fluxo de trabalho moderno, seguro para a privacidade dos dados e financeiramente sustentável.